- Things to note
  - Code must be written sequentially
  - Code is processed from left to right (important to know when assigning variables)
  - Text in quotes (either ‘ ‘ or “ “) is a string.
  - Whitespace is any space or newline in the program.
  - Bits are only 0s and 1s, and they represent instructions or data.
  - Computer switches have gotten extremely small.
  - High-level programs are written and compiled (with a compiler) into an executable program so programmers can run it.
  - Always aim for **abstraction** - the process or result of reducing information and detail in order to simplify a problem or generalize the problem to other, similar problems.
  - Abstraction is why binary code like 1001000 1101001 0100001 01001011-00000000-10000010 becomes Hi! when abstracted.
  - High-level language = high abstraction
    Assembly language = medium abstraction
    Binary language = low abstraction
  - Switch on = 1, switch off = 0
  - ASCII is very cool!
  - IDE’s are also cool, and to write Python, you can write it anywhere.
  - Every Python object–or value automatically created by the interpreter when executing a code–has a value (string or number), type (int or string), and identifier.
  - Strings and integers are immutable, unchangeable objects. And objects are any value created by the interpreter when executing a line of code
  - float is used for floating-point numbers. Don’t use double like you do in C++!
  - Manipulating floating points is as easy as print(f’{myFloat:.2f}’)
    - print(f'{(wall_area / 350.0):.5f}')
  - Python uses PI for pi.
  - + adds, - subtracts or negates, * multiplies, / divides, and ** is used for exponents (such as x ** y == x^y)!
  - Precedence order:
    - ( ) (parentheses)
    - exponent **
    - unary - (negation)
    - * / %
    - + -
    - Left to right evaluation
  - There is always a new line printed after a print() statement. Don’t put endl;!
  - Python also uses compound operators (+=, -=. *=, /=, %=). How convenient!

- / returns a floating-point number. // is used for floor division, returning either an integer or floating-point value based on the divisor and dividend.
    - num_kiloseconds = total_seconds // 1000
    - num_hectoseconds = (total_seconds % 1000) // 100
    - num_seconds = (total_seconds % 1000) % 100
- Module: a file containing Python code that can be used by other modules or scripts. It is made available by using the import statement, and then any object defined can be accessed with dot notation.
- When a module is imported, it is executed immediately.
- if __name__ == '__main__' (...
  This is used to check if the module is being simply imported or run as a script.
- import math (not #include <cmath>!) for math-related things like math.sqrt() and math.pow() and such!
- Converting integers to floats will round them down!
- **Container:** a construct used to group related values together and contains references to other objects instead of data.
    - **List:** a container created by surrounding a sequence of variables or literals with brackets [ ]. Ex: my_list = [10, 'abc'] creates a new list variable my_list that contains the two items: 10 and 'abc'.
        - **Element:** a list item.
        - They are mutable, meaning that the elements in a list can be replaced, reordered, or removed.
        - Each item has an index, starting with item 1 having an index of 0.
- The ***in*** and ***not in*** operators, known as ***membership operators***, yield True or False if the left operand matches the value of an element in the right operand, which is always a container. It can also return matching substrings inside of strings, but it will not return values–only keys–inside of dictionaries.
    - Sometimes a programmer wants to determine whether two variables are the same object. The programmer can use the ***identity operator***, ***is***, to check whether two operands are bound to a single object. The inverse identity operator, ***is not***, gives the negated value of "is". Thus, if x is y is True, then x is not y is False.
- Precedence Rules:
    - ( )
    - * / + -
    - < <= == >= > !=
    - not
    - and
    - or

- A ***code block*** is a series of statements grouped together. A code block in Python is defined by its indentation level. Ex: the number of blank columns from the left edge. The initial code block is not indented. A new code block can follow a statement that ends with a colon, such as an "if" or "else". In addition, a new code block must be more indented than the previous code block.
- **Conditional expression:** has the form
    - expre_when_true if condition else expre_when_false
- All three operands are expressions. The condition in the middle is evaluated first. If the condition evaluates to True, then expr_when_true is evaluated. If the condition evaluates to False, then expr_when_false is evaluated. For example, if x is 2, then the conditional expression 5 if x==2 else 9*x evaluates to 5.
- A conditional expression has three operands and thus is sometimes referred to as a ***ternary operation***.
- A ***loop*** is a program construct that repeatedly executes the loop's statements (known as the ***loop body***) while the loop's expression is true; when the expression is false, execution proceeds past the loop. Each time through a loop's statements is called an ***iteration***.
- A ***while loop*** is a construct that repeatedly executes an indented block of code (known as the ***loop body***) as long as the loop's expression is True. At the end of the loop body, execution goes back to the while loop statement and the loop expression is evaluated *again*. If the loop expression is True, the loop body is executed again. But, if the expression evaluates to False, then execution instead proceeds to below the loop body. Each execution of the loop body is called an ***iteration***, and looping is also called *iterating*.
    - An ***infinite loop*** is a loop that will always execute because the loop's expression is always True. *A common error is to accidentally create an infinite loop by assuming equality will be reached. Good practice is to include greater than or less than along with equality in a loop expression to help avoid infinite loops.*
- The following example uses the statement while user_value != 'q': to allow a user to end a face-drawing program by entering the character 'q'. The letter 'q' in this case is a ***sentinel value***, a value that when evaluated by the loop expression causes the loop to terminate.
- The programmer can use a variable to count the number of iterations, called a ***loop variable***.
- A **for loop** statement loops over each element in a container one at a time, assigning a variable with the next element that can then be used in the loop body. The container in the for loop statement is typically a list, tuple, or string. Each iteration of the loop assigns the name given in the for loop statement with the next element in the container.

- A ***nested loop*** is a loop that appears as part of the body of another loop. The nested loops are commonly referred to as the ***outer loop*** and ***inner loop***.
- A ***break*** statement in a loop causes the loop to exit immediately. A break statement can sometimes yield a loop that is easier to understand.
- A ***continue*** statement in a loop causes an immediate jump to the while or for loop header statement. A continue statement can improve the readability of a loop.
- ***Unpacking*** is a process that performs multiple assignments at once, binding comma-separated names on the left to the elements of a sequence on the right. Ex: num1, num2 = [350, 400] is equivalent to the statements num1 = 350 and num2 = 400.

- Functions
  - print() - outputs text and strings
  - input() - takes input from user
  - AND, OR, or NOT logic gates
  - type() gives type, and id() gives identity (some numerical identifier).
  - len() gives you the length of a string (number of characters, don't think of index numbers!)
  - If…else and if…elseif…else statements are a thing!
    - = is not equal to == when being used in said statements!
    - if statements also look like this!
      - if real_thing == 5:
        - print('This is how you do it!')
      - else:
        - print('This is also how you do it!')
    - elif… is used for else-if stuff!
  - Access a single character in a string literal with brackets!
    - alphabet = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
    - print(alphabet[0]) displays "A" on the screen once executed.
  - **Method:** instructs an object to perform some action, and then is executed by specifying the method name following a "." symbol and an object.
    - **append():** Adds value to the end of a list. Ex: my_list.append('abc')
    - **pop():** Removes the element at index i from list. Ex: my_list.pop(1)
    - **remove():** Removes the first element whose value is v. Ex: my_list.remove('abc')
    - **len():** Find the length of the list.
    - **list1 + list2:** Produce a new list by concatenating list2 to the end of list1.
    - **min():** Find the element in the list with the smallest value. All elements must be of the same type.
    - **max():** Find the element in the list with the largest value. All elements must be of the same type.
    - **sum():** Find the sum of all elements (numbers only).

    - **list.index():** Find the index of the first element in the list whose value matches what is in the parentheses.
    - **list.count():** Count the number of occurrences of the value in the parentheses in the list.
- **Tuple:** stores a collection of data, like a list, but is immutable–once created, the tuple's elements cannot be changed. A tuple is also a sequence type, supporting len(), indexing, and other sequence functions. A new tuple is generated by creating a list of comma-separated values, such as 5, 15, 20. Typically, tuples are surrounded with parentheses, as in (5, 15, 20). Note that printing a tuple always displays surrounding parentheses. A tuple is not as common as a list in practical usage but can be useful when a programmer wants to ensure that values do not change. Tuples are typically used when element position, and not just the relative ordering of elements, is important
- **Named Tuple:** allows the programmer to define a new simple data type that consists of named attributes. A Car named tuple with fields like Car.price and Car.horsepower would more clearly represent a car object than a list with index positions correlating to some attributes. The ***namedtuple*** container must be imported to create a new named tuple.
- A ***dictionary*** is a Python container used to describe associative relationships. A dictionary is represented by the ***dict*** object type. A dictionary associates (or "maps") keys with values. A ***key*** is a term that can be located in a dictionary, such as the word "cat" in the English dictionary. A ***value*** describes some data associated with a key, such as a definition. A key can be any immutable type, such as a number, string, or tuple; a value can be any type.
    - A dict object is created using ***curly braces*** { } to surround the ***key:value pairs*** that comprise the dictionary contents. Ex: players = {'Lionel Messi': 10, 'Cristiano Ronaldo': 7} creates a dictionary called players with two keys: 'Lionel Messi' and 'Cristiano Ronaldo', associated with the values 10 and 7 (their respective jersey numbers). An empty dictionary is created with the expression players = { }.
    - Dictionaries are typically used in place of lists when an associative relationship exists. Ex: If a program contains a collection of anonymous student test scores, those scores should be stored in a list. However, if each score is associated with a student name, a dictionary could be used to associate student names to their score.
    - Though dictionaries maintain a left-to-right ordering, dictionary entries cannot be accessed by indexing. To access an entry, the key is specified in brackets [ ]. If no entry with a matching key exists in the dictionary, then a ***KeyError*** runtime error occurs and the program is terminated.

- A dictionary is mutable, so entries can be added, modified, and deleted as necessary by a programmer. A new dictionary entry is added by using brackets to specify the key: prices['banana'] = 1.49. A dictionary key is unique; attempting to create a new entry with a key that already exists in the dictionary *replaces* the existing entry. The ***del*** keyword is used to remove entries from a dictionary: del prices['papaya'] removes the entry whose key is 'papaya'. If the requested key to delete does not exist, then a KeyError occurs.
    - dict[k] = v: Adds the new key-value pair k-v, if dict[k] does not already exist.
      Example: students['John'] = 'A+'
    - dict[k] = v: Updates the existing entry dict[k], if dict[k] already exists.
      Example: students['Jessica'] = 'A+'
    - del dict[k]: Deletes the entry dict[k].
      Example: del students['Rachel']

➢ A ***set*** is an unordered collection of unique elements. A set has the following properties:

- Elements are unordered: Elements in the set do not have a position or index.
- Elements are unique: No elements in the set share the same value.
- A set can be created using the ***set()*** function, which accepts a sequence-type iterable object (list, tuple, string, etc.) whose elements are inserted into the set. A ***set literal*** can be written using curly braces { } with commas separating set elements. Note that an empty set can only be created using set().
- **Modifying sets**
    - Sets are mutable – elements can be added or removed using set methods. The ***add()*** method places a new element into the set if the set does not contain an element with the provided value. The ***remove()*** and ***pop()*** methods remove an element from the set.
    - Additionally, sets support the len() function to return the number of elements in a set. To check if a specific value exists in a set, a membership test such as value in set (discussed in another section) can be used.
    - **len(set)** - Find the length (number of elements) of the set.
    - **set1.update(set2)** - Adds the elements on set2 in set1.
    - **set.add(value)** - Adds value into the set.
    - **set.remove(value)** - Removes value from the set. Raises KeyError if value is not found.

- **set.pop()** - Random a random element from the set.
- **set.clear()** - Clears all elements from the set.
- **set.intersection(set_a, set_b, set_c,...)** - Returns a new set containing only the elements in common between *set* and all provided sets.
- **set.union( )** - Returns a new set containing all of the unique elements in all sets.
- **set.difference( )** - Returns a set containing only the elements of *set* that are not found in any of the provided sets.
- **set_a.symmetric_difference(set_b)** - Returns a set containing only elements that appear in exactly one of *set_a* or *set_b*.

➢ You can do operation chaining in Python and only Python!

```
if 19 <= x < 21:
    print("You stupid")
```

➢ The ***randint()*** function provides a new random number each time the function is called. The arguments to randint(), 0 and 2, provide the minimum and maximum values that the function may return.

➢ A for loop may also iterate backward over a sequence, starting at the last element and ending with the first element, by using the ***reversed()*** function to reverse the order of the elements.

```
state_dict = {
    'Indiana': 'IN',
    'Connecticut': 'CT',
    'Idaho': 'ID',
    'Illinois': 'IL'
}

state_name = input()
state_code = input()
state_dict[state_name] = state_code

for state_name in state_dict:
    print(f"{state_name}'s code: {state_dict[state_name]}")

((Reads out the dictionary above))
```

➢ ***range()*** generates a sequence of integers between a starting integer that is included in the range, an ending integer that is not included in the range, and an integer step value. The sequence is generated by starting at the start integer and incrementing by the step value until the ending integer is reached or surpassed.

```
range(starting value, exclusive ending value, incrementation)
```

- The ***enumerate()*** function retrieves both the index and corresponding element value at the same time, providing a cleaner and more readable solution.
    - `origins = [4, 8, 10]`
    -
    - `for (index, value) in enumerate(origins):`
    - `    print(f'Element {index}: {value}')`
- Dictionaries (key:value), when put in for loops like this will print the key.
    - rentals = {
    - 'skis' : 20.00,
    - 'boots' : 10.00,
    - 'skates' : 4.00
    - }
    - for x in rentals:
    - print(x, end=' ')

❖ Editing

- end=' ' - ends printed text with a single space instead of a newline
- \n - creates new line
- int(input()) - makes input into an integer that can be used for calculations
- float(input()) - same thing as above but for floating point stuff
- \t - adds a tab
- \...\ - prints \ inside the \...\ without breaking the string
- \' - allows ' to be used in a string without messing anything up
- Anything between '...' or "..." is a string literal! Even numbers!
- doing my_variable[-1] will access the last character of the my_variable string.
- print(var, sentence, something) will display each variable with automatic single spacing between them. Don't accommodate the code!
- Concatenation = print(x + y + z) while x, y, and z are string literals
- print(f"...{variable1}...") is a formatted string literal with expressions/replacement fields where its value replaces the expression.
    - Having a replacement field with a = at the end of it will output the result of the expression it's attached to.
- printing long strings requires
- Str = ("..."
- "...")
- / divides normally, // divides and rounds down.
- Whitespace is counted in string variable indexes.
- print(chr(65)) prints 'A'; 65 is code point for 'A'.
- print(sys.argv) displays the variable argv in the sys module.
- Dividing by zero is a runtime error.

- ➢ Logic errors don't crash programs; they're simple flaws in what you want vs. what the computer sees.
- ➢ print('Food', end='') will print 'Food' without a new line.
- ➢ ***docstring***: a multi-line string literal delimited at the beginning and end by triple quotes. Use either single (') or double (") quotes.

❖ Computers

- ➢ A computer has a processor that reads instructions stored in its memory.
- ➢ Computers consist of an **input/output device** (keyboard and screen/monitor), a **disk or hard drive** (which acts as a non-volatile storage with slower access), **RAM or random-access memory** (which is volatile storage with faster access usually located off the processor chip), a **processor, CPU (central processing unit), or microprocessor** (which runs everything including the operating system of the computer, and contains a little RAM called cache memory), and a **clock** (which executes one instruction per clock tick).
- ➢ Moore's Law comes in the form of IC (integrated circuit) capacity doubling every 18 months.
- ➢ A CPU is the computer's hardware "brain" component that consists of the arithmetic logic unit (which performs arithmetic (+, -, *, /) and logic (and, or, xor, not) operations on binary integers) and a control unit (which tells other components how to respond to computer instructions).

Because in programming = means assignment, a variable may appear on both the left and right as in x = x + 1. If x was originally 6, x is assigned with 6 + 1, or 7. The statement overwrites the original 6 in x.

Increasing a variable's value by 1, as in x = x + 1, is common and known as ***incrementing*** the variable.